

07. 딥러닝을 시작합니다.

1. 텐서플로와 케라스 (딥러닝 준비)

개념 정리

- **텐서플로(TensorFlow)**: 구글이 만든 딥러닝 라이브러리 (CPU + GPU 사용)
- **케라스(Keras)**: 원래 독립 딥러닝 라이브러리였지만, 이제는 **텐서플로의 고수준 API**로 통합
 - `tf.keras` 이름으로 사용
 - 모델을 "층을 쌓듯이" 간단하게 만드는 데 초점
- GPU를 잘 활용해서 행렬/벡터 연산이 많은 신경망을 빠르게 학습할 수 있는 게 특징입니다.

2. 7-1 인공 신경망

2-1. 패션 MNIST 데이터 불러오기

패션 MNIST: 28×28 회색조 이미지, 10종류 패션 아이템(티셔츠, 바지, 가방, 신발 등) 분류 문제.

```
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np

# 1) 데이터 적재
(train_input, train_target), (test_input, test_target) = \
    keras.datasets.fashion_mnist.load_data()

print(train_input.shape, train_target.shape) # (60000, 28, 28) (60000,)
print(test_input.shape, test_target.shape)   # (10000, 28, 28) (10000,)

# 2) 샘플 이미지 10개 확인
fig, axes = plt.subplots(1, 10, figsize=(10, 1))
for i, ax in enumerate(axes):
    ax.imshow(train_input[i], cmap="gray_r")
    ax.axis("off")
plt.show()

# 3) 레이블 분포 확인
print(np.unique(train_target, return_counts=True))
```

→ 0~9 레이블 각각 6,000개씩, 총 60,000개 훈련 샘플입니다.

2-2. 로지스틱 회귀로 baseline 만들기

신경망으로 가기 전에, 이전 장에서 배운 ****로지스틱 회귀(SGDClassifier)****로 먼저 성능을 확인합니다.

이미지는 28×28이라서, 로지스틱 회귀에 넣으려면 ****1차원 벡터(784차원)****로 펼쳐야 합니다.

```

from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import cross_validate

# 1) 0~255 → 0~1 스케일링
train_scaled = train_input / 255.0

# 2) (60000, 28, 28) → (60000, 784)
train_scaled_flat = train_scaled.reshape(-1, 28 * 28)

# 3) 로지스틱 회귀 모델 (확률적 경사 하강법)
sgd = SGDClassifier(loss="log_loss", max_iter=5, random_state=42)

scores = cross_validate(sgd, train_scaled_flat, train_target,
                        n_jobs=-1, return_train_score=True)

print("평균 검증 정확도:", scores["test_score"].mean())

```

이렇게 만든 모델은 **기본적인 baseline**이고, 7장에서는 “이보다 더 잘 해보자”가 목표입니다.

2-3. 인공 신경망(ANN) 기본 구조

로지스틱 회귀와 가장 단순한 인공 신경망은 거의 같습니다. 차이점은 다음처럼 “**신경망**”이라고 부르는 구조와 용어를 붙여서 생각한다는 점입니다.

- 입력층: 784개의 픽셀 값
- 출력층: 10개의 뉴런 (10개 클래스의 점수/확률)
- 각 뉴런: $z = w \cdot x + b$ 계산 (선형결합)
- 출력층에 ****소프트맥스(softmax)****를 씌워 각 클래스를 확률처럼 해석

이런 선형계산을 수행하는 층을 케라스에서는 **Dense(밀집층)** 라고 부릅니다.

2-4. 케라스로 인공 신경망 모델 만들기

- *1층짜리 신경망(입력 → 출력층만 있는 모델)*부터 만들어 봅니다.

```

from sklearn.model_selection import train_test_split

# 1) 다시 스케일링 + 평탄화
train_scaled = train_input / 255.0
train_scaled = train_scaled.reshape(-1, 28 * 28)

# 2) 훈련/검증 세트 분리 (20%를 검증용으로)
train_scaled, val_scaled, train_target, val_target = train_test_split(
    train_scaled, train_target, test_size=0.2, random_state=42
)

# 3) 출력층만 있는 간단한 신경망
model = keras.Sequential([

```

```

keras.layers.Dense(10,      # 뉴런 10개 (클래스 개수)
                    activation="softmax",
                    input_shape=(784,))
])

# 4) 모델 컴파일 (손실함수 + 지표 설정)
model.compile(loss="sparse_categorical_crossentropy", # 정수 레이블용 다중분류 CE
              metrics=["accuracy"])

# 5) 훈련
model.fit(train_scaled, train_target, epochs=5)

# 6) 검증 세트 성능
model.evaluate(val_scaled, val_target)

```

여기서 포인트:

- `sparse_categorical_crossentropy`
→ 타깃이 0~9 정수이면 그대로 사용 가능 (원-핫 인코딩 안 해도 됨)
- 만약 타깃을 원-핫으로 바꿨다면 `categorical_crossentropy` 사용

3. 7-2 심층 신경망 (은닉층 추가)

이제 **은닉층(hidden layer)**을 추가해서 "심층 신경망(Deep Neural Network)"을 만듭니다.

3-1. 2층(은닉층 1개) 신경망

은닉층에 **100개 뉴런**, 활성화 함수로 **sigmoid**를 먼저 사용해 봅니다.

```

from tensorflow import keras
from sklearn.model_selection import train_test_split

(train_input, train_target), (test_input, test_target) = \
    keras.datasets.fashion_mnist.load_data()

# 1) 스케일링 + 평탄화
train_scaled = train_input / 255.0
train_scaled = train_scaled.reshape(-1, 28 * 28)

train_scaled, val_scaled, train_target, val_target = train_test_split(
    train_scaled, train_target, test_size=0.2, random_state=42
)

# 2) 은닉층 1개 + 출력층
hidden = keras.layers.Dense(100, activation="sigmoid", input_shape=(784,))
output = keras.layers.Dense(10, activation="softmax")

model = keras.Sequential([hidden, output])

```

```
model.summary()
```

요약에서 볼 수 있는 것:

- **Dense** 층마다 **출력 형태와 파라미터(가중치+절편) 개수**가 표시됨
- 전체 파라미터 수 = 모든 층의 파라미터 수 합

훈련은 7-1과 동일하게:

```
model.compile(loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_scaled, train_target, epochs=5)
model.evaluate(val_scaled, val_target)
```

→ 은닉층을 하나 넣었을 뿐인데, 보통 **정확도가 눈에 띄게 향상**됩니다.

3-2. 다른 방식으로 층 쌓기 (add / 리스트)

같은 모델을 만드는 문법은 여러 가지가 있습니다.

```
# 1) Sequential([...])에 바로 층 객체 나열
model = keras.Sequential([
    keras.layers.Dense(100, activation="sigmoid", input_shape=(784,)), name="hidden"),
    keras.layers.Dense(10, activation="softmax", name="output")
], name="fashion_mnist_model")

# 2) 나중에 add()로 층을 하나씩 추가
model2 = keras.Sequential(name="fashion_mnist_model_2")
model2.add(keras.layers.Dense(100, activation="sigmoid", input_shape=(784,)))
model2.add(keras.layers.Dense(10, activation="softmax"))
```

3-3. 렐루(ReLU) 활성화 함수

초창기에는 은닉층에 시그모이드를 많이 썼지만, **층이 깊어질수록 기울기 소실** 때문에 학습이 잘 안 되는 문제가 큼니다.

그래서 지금은 ****ReLU ($\max(0, z)$)****가 기본 활성화 함수처럼 쓰입니다.

3-4. Flatten 층으로 입력 펼치기

지금까지는 **reshape()** 로 28×28을 784로 바꿨지만, 케라스에는 이를 자동으로 해주는 **Flatten** 층이 있습니다.

```
# 다시 원본 이미지 모양(28, 28) 그대로 사용
(train_input, train_target), (test_input, test_target) = \
    keras.datasets.fashion_mnist.load_data()

train_scaled = train_input / 255.0

train_scaled, val_scaled, train_target, val_target = train_test_split(
```

```

train_scaled, train_target, test_size=0.2, random_state=42
)

model = keras.Sequential()
model.add(keras.layers.Flatten(input_shape=(28, 28))) # (28,28) → (784,)
model.add(keras.layers.Dense(100, activation="relu")) # 은닉층
model.add(keras.layers.Dense(10, activation="softmax")) # 출력층

model.summary()

```

- **Flatten** 층은 **학습할 파라미터가 0개**이고, 그저 입력을 일렬로 펼치는 역할만 합니다.
- 첫 Dense 층의 **input_shape** 를 **Flatten** 쪽으로 옮겨 간 느낌입니다.

훈련도 그대로:

```

model.compile(loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_scaled, train_target, epochs=5)
model.evaluate(val_scaled, val_target)

```

→ sigmoid보다 약간 더 좋은 성능이 보통 나옵니다.

3-5. 옵티마이저(Optimizer) 개념 & 코드

옵티마이저: 신경망의 가중치/절편을 어떻게 업데이트할지 결정하는 알고리즘.

대표적인 것들:

- **SGD** (확률적 경사 하강법)
- **SGD(momentum=0.9, nesterov=True)** → 모멘텀 + 네스테로프 모멘텀
- **Adagrad**, **RMSprop** (적응적 학습률)
- **Adam** (모멘텀 + RMSprop 장점 결합)

```

# 1) SGD (기본)
model.compile(optimizer="sgd",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 2) 모멘텀 + 네스테로프
sgd = keras.optimizers.SGD(learning_rate=0.01,
                             momentum=0.9,
                             nesterov=True)
model.compile(optimizer=sgd,
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 3) Adam (많이 쓰는 기본값)
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",

```

```

metrics=["accuracy"])
model.fit(train_scaled, train_target, epochs=5)
model.evaluate(val_scaled, val_target)

```

실습에서는 **RMSprop, Adam**이 기본 SGD보다 보통 더 좋은 결과를 줍니다.

4. 7-3 신경망 모델 훈련

이 절에서는 “훈련 과정을 좀 더 깊게 들여다보기”가 핵심입니다.

4-1. History 객체로 손실/정확도 곡선 그리기

`model.fit()` 은 **History** 객체를 반환하고, 여기에 에포크별 손실/지표 값이 딕셔너리 형태로 들어 있습니다.

```

from tensorflow import keras
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

(train_input, train_target), (test_input, test_target) = \
    keras.datasets.fashion_mnist.load_data()

train_scaled = train_input / 255.0
train_scaled, val_scaled, train_target, val_target = train_test_split(
    train_scaled, train_target, test_size=0.2, random_state=42
)

def build_model(extra_layer=None):
    model = keras.Sequential()
    model.add(keras.layers.Flatten(input_shape=(28, 28)))
    model.add(keras.layers.Dense(100, activation="relu"))
    if extra_layer is not None:
        model.add(extra_layer)
    model.add(keras.layers.Dense(10, activation="softmax"))
    return model

model = build_model()
model.compile(loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_scaled, train_target,
                    epochs=5, verbose=0)

print(history.history.keys()) # dict에 들어있는 키들

# 손실 곡선
plt.plot(history.history["loss"])
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()

```

```
# 정확도 곡선
plt.plot(history.history["accuracy"])
plt.xlabel("epoch")
plt.ylabel("accuracy")
plt.show()
```

4-2. 검증 손실로 과대적합/과소적합 확인

훈련 손실만 보면 안 되고, **검증 손실/정확도**도 같이 봐야 모델이 과대적합인지 알 수 있습니다.

```
model = build_model()
model.compile(loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_scaled, train_target,
                    epochs=20, verbose=0,
                    validation_data=(val_scaled, val_target))

print(history.history.keys()) # 'val_loss', 'val_accuracy' 추가됨

plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])
plt.xlabel("epoch")
plt.ylabel("loss")
plt.legend(["train", "val"])
plt.show()
```

전형적인 패턴:

- 훈련 손실: 계속 감소
- 검증 손실: 어느 시점까진 감소하다가 다시 증가 → **그 이후 에포크부터 과대적합**

Adam으로 바꾸면 검증 손실 곡선이 더 안정적으로 줄어들기도 합니다.

```
model = build_model()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_scaled, train_target,
                    epochs=20, verbose=0,
                    validation_data=(val_scaled, val_target))

plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])
plt.xlabel("epoch")
plt.ylabel("loss")
```

```
plt.legend(["train", "val"])
plt.show()
```

4-3. 드롭아웃(Dropout)으로 과대적합 줄이기

드롭아웃: 훈련 중에 은닉층의 일부 뉴런 출력을 랜덤하게 0으로 만들어 특정 뉴런에 과도하게 의존하는 것을 막는 기법입니다.

```
dropout_layer = keras.layers.Dropout(0.3) # 출력의 30%를 랜덤하게 끄

model = build_model(extra_layer=dropout_layer)
model.summary() # Dropout 층은 파라미터 0개

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_scaled, train_target,
                   epochs=20, verbose=0,
                   validation_data=(val_scaled, val_target))

plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])
plt.xlabel("epoch")
plt.ylabel("loss")
plt.legend(["train", "val"])
plt.show()
```

- 드롭아웃은 **훈련 중에만 적용**되고, 평가/예측할 때는 자동으로 꺼집니다.

4-4. 모델 저장과 복원

훈련 잘 된 모델을 나중에 다시 쓰고 싶으니까, **저장/복원** 방법도 배웁니다.

```
model = build_model(extra_layer=keras.layers.Dropout(0.3))
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_scaled, train_target,
                   epochs=10, verbose=0,
                   validation_data=(val_scaled, val_target))

# 1) 가중치만 저장
model.save_weights("model-weights.h5")
```



```
# 2) 모델 구조 + 가중치 통째로 저장
model.save("model-whole.h5")
```

다시 사용할 때:

```
import numpy as np

# 1) 같은 구조의 새 모델을 만든 뒤 가중치만 로드
model1 = build_model(extra_layer=keras.layers.Dropout(0.3))
model1.load_weights("model-weights.h5")

val_pred = model1.predict(val_scaled)
val_labels = np.argmax(val_pred, axis=-1)
print("검증 정확도(가중치 로드):",
      np.mean(val_labels == val_target))

# 2) 통째로 저장한 모델을 바로 로드
model2 = keras.models.load_model("model-whole.h5")
model2.evaluate(val_scaled, val_target)
```

4-5. 콜백(Callback), ModelCheckpoint, EarlyStopping

콜백(callback): 훈련 도중에 특정 작업을 수행하게 해 주는 "훅(hook)" 같은 객체입니다.

대표 예시: **ModelCheckpoint**(최고 성능 모델 자동 저장), **EarlyStopping**(조기 종료).

(1) ModelCheckpoint – 최고 검증 성능일 때만 저장

```
checkpoint_cb = keras.callbacks.ModelCheckpoint(
    "best-model.h5",
    save_best_only=True # 검증 손실이 가장 낮을 때만 저장
)

model = build_model(extra_layer=keras.layers.Dropout(0.3))
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

model.fit(train_scaled, train_target,
          epochs=20, verbose=0,
          validation_data=(val_scaled, val_target),
          callbacks=[checkpoint_cb])

# 훈련이 끝난 뒤 best-model.h5 로드
best_model = keras.models.load_model("best-model.h5")
best_model.evaluate(val_scaled, val_target)
```

(2) EarlyStopping – 검증 성능이 더 이상 좋아지지 않으면 정지

```

checkpoint_cb = keras.callbacks.ModelCheckpoint(
    "best-model.h5",
    save_best_only=True
)
early_stopping_cb = keras.callbacks.EarlyStopping(
    patience=2,          # 개선 안 되는 에포크 2번까지 참았다가
    restore_best_weights=True # 가장 좋았던 가중치로 자동 복원
)

model = build_model(extra_layer=keras.layers.Dropout(0.3))
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_scaled, train_target,
                   epochs=20, verbose=0,
                   validation_data=(val_scaled, val_target),
                   callbacks=[checkpoint_cb, early_stopping_cb])

print("멈춘 에포크 인덱스:", early_stopping_cb.stopped_epoch)

# 손실 곡선 확인
import matplotlib.pyplot as plt

plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])
plt.xlabel("epoch")
plt.ylabel("loss")
plt.legend(["train", "val"])
plt.show()

# 최종 검증 성능
model.evaluate(val_scaled, val_target)

```

EarlyStopping 덕분에:

- **에포크 수를 넉넉하게 크게 잡아도**
- 검증 손실이 더 나빠지면 알아서 훈련을 멈추고
- **가장 좋은 시점의 가중치로 복원해 줍니다.**